Model Graph Visualization with Transformation Passes

1. Introduction

The Neural Network Compression Framework (NNCF) provides algorithms for optimizing neural network models across different frameworks, including OpenVINO, ONNX, and PyTorch.

Many of these algorithms analyze or modify a model's computational graph. When developing and debugging such algorithms, it is often useful to visualize the graph to better understand its structure, inspect connections between operations, and analyze the effects of graph transformations.

Existing visualization tools, such as Netron, make it possible to inspect neural network models. However, **large computational graphs can be difficult to read and understand**, especially when they contain thousands of operations and connections.

For example, consider the TinyLlama-1.1B-Chat-v1.0 model. You can export it to OpenVINO IR format using Optimum Intel:

```
optimum-cli export openvino \
    --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \
    --weight-format fp16 \
    OUTPUT_DIR
```

Open the exported `OUTPUT_DIR/openvino_model.xml` in Netron and inspect the resulting graph. You may notice that the large number of operations, auxiliary nodes, and connections makes it difficult to follow the main computation flow or identify specific graph patterns.

One possible solution is to **simplify the graph's visual representation**. For example, `Constant` nodes representing operation weights could be hidden, with their data types and shapes displayed as attributes of the corresponding operations. Other improvements might include highlighting selected nodes or collapsing sequences of operations into single visualization nodes.

The goal of this assignment is to explore this approach by developing a lightweight, extensible graph visualization tool that allows users to customize how computational graphs are displayed.

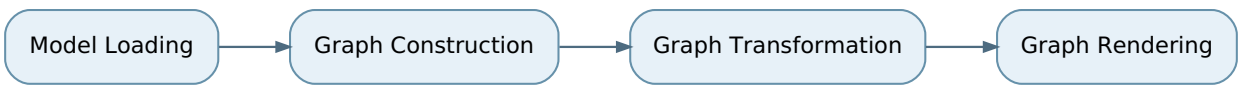
2. Task Overview

Your task is to develop a Python library for visualizing neural network computational graphs, along with a command-line tool built on top of the library's API.

The tool should load a neural network model, convert it into a framework-independent graph representation, apply a configurable sequence of visualization transformations, and export the resulting graph as an SVG file that can be opened in a web browser.

At a high level, the processing pipeline consists of four steps:

1. **Model Loading** — Load a neural network model in OpenVINO format.
2. **Graph Construction** — Convert the model into a framework-independent intermediate graph representation.
3. **Graph Transformation** — Apply a configurable sequence of transformation passes to simplify or customize the visualization.
4. **Graph Rendering** — Render the transformed graph and save it as an SVG file.



The solution should provide both a **Python API** and a **command-line interface (CLI)** that uses this API.

For example, the CLI might support the following commands:

```
# Visualize a model without transformations
python model_graph_viewer.py --model model.xml --output graph.svg

# Visualize a model with transformation passes
python model_graph_viewer.py --model model.xml --output simplified.svg \
    --passes hide-constants,collapse-patterns
```

These commands are provided for illustration only. You are free to design your own interfaces and choose appropriate option names.

An important goal of this assignment is extensibility. The architecture should make it easy to add new transformation passes and support additional model formats without modifying the existing components.

The following sections describe the requirements for each component in more detail.

3. Implementation Requirements

This section describes the functional and technical requirements for each component of the visualization framework. You are free to choose the internal architecture, data structures, and APIs, provided that your implementation satisfies the requirements below.

3.1. Model Loading

Implement a component that loads a neural network model and converts it into a framework-independent intermediate graph representation.

Requirements:

- Support loading models in OpenVINO IR format (`.xml` and associated `.bin` files).
- Extract the model's operations, connections, input/output ports, and available tensor metadata.
- Keep model-specific parsing logic separate from the intermediate graph representation, transformation passes, and renderer.
- Handle unsupported or invalid input files with meaningful error messages.

> Important: Although this assignment focuses primarily on OpenVINO support, the architecture should make it easy to add support for other model formats, such as ONNX, without modifying existing transformation passes or rendering logic.

3.2. Intermediate Graph Representation

Design a framework-independent representation of a computational graph. Each node should represent an operation (e.g., `MatMul`, `Add`, or `Split`), and each edge should represent a connection between two operation ports.

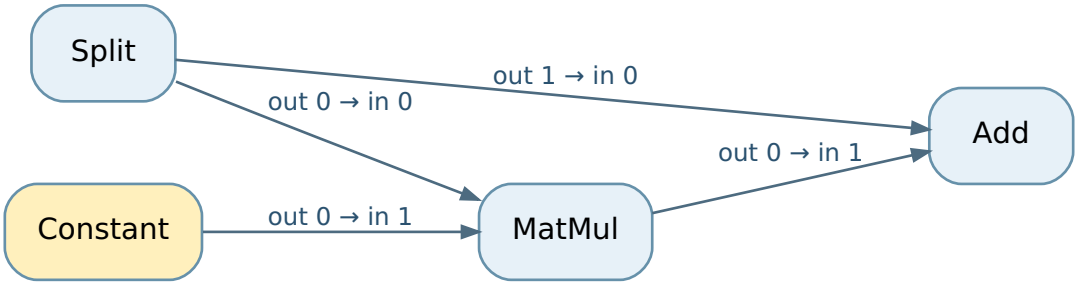
The graph should store at least the following information:

Node: - Unique identifier, separate from the node name. - Node name and operation type. - Operation attributes. - Optional visualization metadata.

Edge: - Source node and output port index. - Destination node and input port index. - Tensor shape and data type, if available.

> Important: Knowing that two nodes are connected is not enough. The graph must preserve which output port of the source operation connects to which input port of the destination operation.

For example:



The implementation should correctly handle:

- Operations with multiple inputs and outputs.
- Multiple connections between the same pair of nodes.
- Nodes with duplicate names.
- Dynamic or partially known tensor shapes.
- Missing tensor metadata.
- Model input and output operations.

The representation should support graph modifications while preserving the validity of remaining nodes and connections.

3.3. Graph Transformation Pipeline

Implement an extensible mechanism for applying a configurable sequence of transformation passes to the intermediate graph.

Conceptually, the API might look like:

```
graph = parser.parse(model)

for transformation_pass in transformation_passes:
    graph = transformation_pass.apply(graph)
```

The exact API is up to you.

Requirements:

- Transformation passes must operate on the intermediate graph representation.
- Passes must be independent of OpenVINO-specific objects.
- Passes must be executed in the specified order.
- It must be possible to enable, disable, and configure individual passes.
- Adding a new pass should not require modifying the parser or renderer.
- Transformations must not modify the original neural network model.

Implement the following transformation passes.

3.3.1. Node Highlighting Pass

Implement a pass that accepts a list of node names and highlights matching nodes by changing their border color to red.

Requirements:

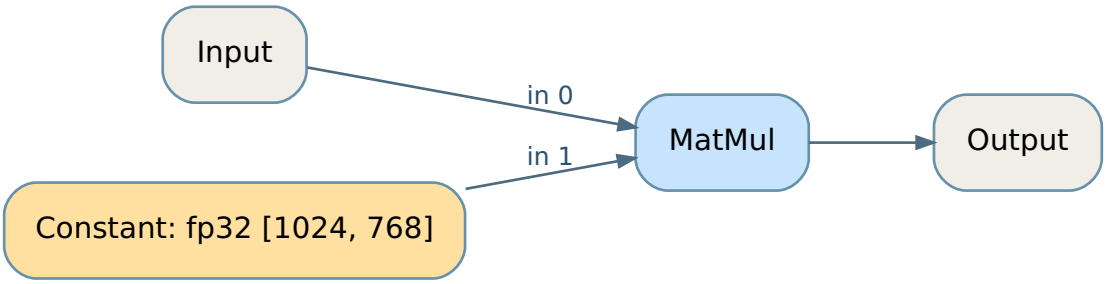
- Highlight all nodes whose names match the provided list.
- Leave other nodes unchanged.
- Store highlighting information as visualization metadata.

3.3.2. Constant Hiding Pass

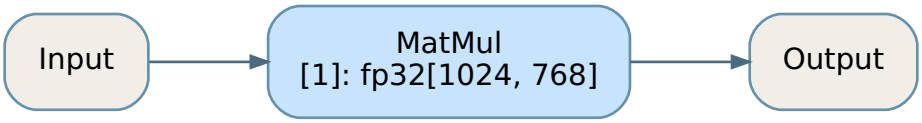
Implement a pass that simplifies the graph by hiding `Constant` nodes and displaying their relevant properties as attributes of the operations that consume them.

For example:

Before transformation:



After transformation:



Requirements:

- Preserve the original destination input port index.
- Display the Constant's data type and shape (e.g., `[1]: fp32[1024, 768]`).
- Correctly handle Constants shared by multiple consumers. Each consumer should receive the corresponding attribute, including a marker indicating that the Constant is shared. The marker should include the Constant's unique node ID, allowing users to identify operations that reference the same Constant (e.g., `[1]: fp32[1024, 768] (shared: <node_id>)`). Constants with only one consumer do not require this marker.
- Remove the hidden Constant connections from the visualization.
- Remove a Constant node only when it has no remaining visible connections.
- Preserve unrelated graph connections.
- Do not display or copy the actual tensor values.

The original model must remain unchanged.

3.3.3. (OPTIONAL) Custom Visualization Improvement Pass

Explore the visualization of the `TinyLlama-1.1B-Chat-v1.0` model in Netron and identify at least one issue that makes the graph difficult to understand. Propose and implement a transformation pass that improves the visualization. For example, you could simplify repetitive structures, group related operations, or reduce unnecessary visual details. Demonstrate your improvement using before-and-after SVG examples and briefly explain why it makes the graph easier to understand. This is an open-ended task with no predefined solution.

3.4. Graph Rendering

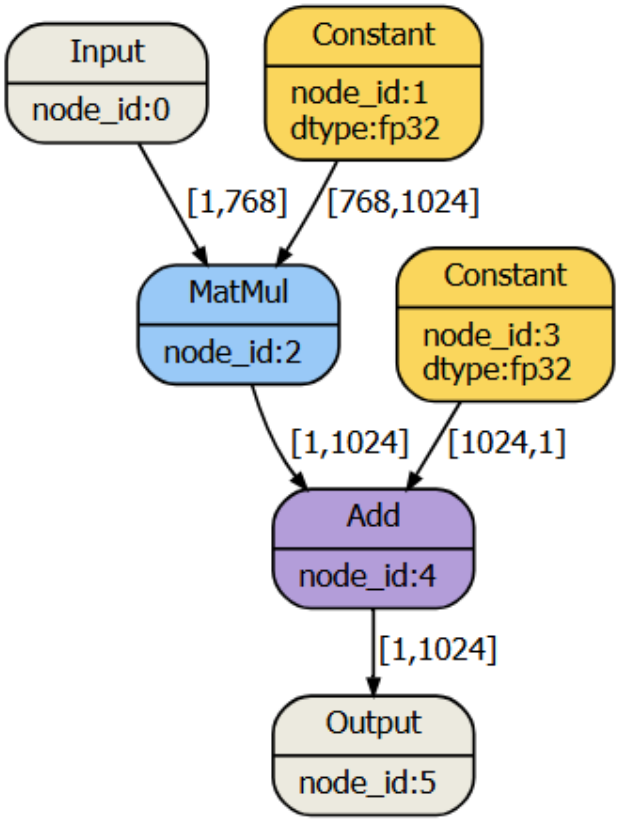
Implement a component that renders the transformed intermediate graph as an SVG file that can be opened in a standard web browser. The visualization should use a layered (hierarchical) layout that makes the computation flow easy to follow. Nodes, directed edges, and relevant operation information should be clearly readable. We recommend using Graphviz, particularly its `dot` layout engine, but other solutions are acceptable if you explain your choice.

To help you get started, we provide a runnable example, `starter_code.py`, that demonstrates basic graph visualization using the Python `graphviz` library.

Run the example with:

```
python starter_code.py
```

This generates a `simple_graph.svg` file in the same directory as the script. Open the file in your browser to see a visualization similar to the following:



You are encouraged to build on this example and improve the visualization. The Graphviz documentation may be helpful, particularly for customizing node styles, using record or HTML-like labels, connecting edges to specific ports, and configuring graph layouts.

Requirements:

- Generate a valid SVG file that can be opened in a standard web browser.
- Use a hierarchical layout suitable for computational graphs, preferably top-to-bottom.
- Clearly display operation types and relevant attributes.
- Show directed connections between operations.
- Preserve input/output port information in the visualization.
- Apply visualization metadata produced by transformation passes, such as node highlighting.
- Support rendering graphs containing operation types not known in advance.

Visual styling is left to you. We value readability and correctness over sophisticated graphics.

3.5. Command-Line Interface and Python API

Your solution must provide both a Python API and a command-line interface (CLI) built on top of that API. Both interfaces should allow users to load a model, apply a configurable sequence of transformation passes, and export the resulting graph as an SVG file.

Requirements:

- Allow users to specify the input model and output SVG file path.
- Support generating a visualization without applying any transformation passes.
- Allow users to select and configure transformation passes, including pass-specific options.
- Apply transformation passes in the order specified by the user.
- Provide clear and meaningful error messages for invalid arguments, unsupported inputs, or other failures.
- Document both interfaces in the README, including setup instructions, usage examples, and a short getting-started guide.

You are free to design your own interfaces and choose appropriate option names, as long as the required functionality is supported.

4. Testing Requirements

Your solution should include unit tests to verify the correctness of the intermediate graph representation and transformation passes. We recommend using pytest, but other testing frameworks are acceptable.

Requirements:

- **Intermediate graph representation** - Test node and edge creation, graph connectivity, and correct handling of input/output port indices. Include edge cases such as multiple connections between the same nodes and dynamic tensor shapes.
- **Model parsing** - Verify that OpenVINO models are correctly converted into the intermediate graph representation, preserving operation types, connections, port indices, and available tensor metadata.
- **Node Highlighting Pass** - Verify that only nodes matching the specified names are highlighted and that other nodes remain unchanged.
- **Constant Hiding Pass** - Test Constants with single and multiple consumers. Verify that the correct attributes are added, shared Constants are marked with their unique IDs, and unrelated graph connections are preserved.
- **Transformation pipeline** - Verify that multiple passes can be applied sequentially in the specified order and that the resulting graph remains valid.
- **Graph rendering** - Verify that a valid SVG file is generated and that visualization metadata, such as node highlighting, is reflected in the output.

Whenever possible, test graph transformations independently of model parsing and rendering. We encourage you to construct small synthetic graphs to test specific scenarios and edge cases. In addition to unit tests, include at least one **end-to-end test** that loads a small OpenVINO model, applies transformation passes, and generates an SVG file. Document how to run the tests in the README.

> Important: You do not need to verify the exact visual appearance of generated SVG files. Focus on testing graph structure, transformation correctness, and the presence of expected elements in the output.

5. Scalability and Performance

Neural network models can contain thousands of operations and connections. Your implementation should be designed to handle reasonably large computational graphs without excessive processing time or memory usage.

Requirements:

- **Scalability** — Aim to support graphs containing approximately 5,000 nodes or more.
- **Efficiency** — Avoid unnecessarily expensive graph operations, such as repeatedly searching through all nodes or edges when more efficient approaches are available.
- **Memory usage** — Avoid unnecessary copies of the entire graph and excessive memory consumption, especially when applying transformation passes.
- **Performance evaluation** — Measure and report the time spent on the main stages of the pipeline:
  - Model loading and intermediate graph construction.
  - Applying transformation passes.
  - Rendering and exporting the graph to SVG.

Test your implementation on at least one reasonably large neural network model, preferably `TinyLlama-1.1B-Chat-v1.0`. Include the number of nodes and edges, execution times, and a brief description of your testing environment in the README.

> Important: We do not expect real-time or interactive rendering performance. Large graphs may take considerable time to render, especially when using Graphviz. We are more interested in efficient graph processing, reasonable design decisions, and your ability to identify and explain performance bottlenecks than in meeting strict execution time limits.

If your implementation encounters performance or visualization limitations with large models, document them and briefly suggest possible improvements.

6. Evaluation Criteria

The assignment will be evaluated based on the following criteria:

- **Software Design** - Clear separation of responsibilities, well-designed abstractions, a suitable intermediate graph representation, and an extensible architecture that makes it easy to add new model formats and transformation passes.
- **Correctness** - Accurate representation of model graphs, correct behavior of transformation passes, and proper handling of edge cases, such as shared Constants, multiple input/output ports, and branching connections.
- **Code Quality** - Readable, well-organized, and maintainable code, meaningful naming, appropriate type annotations, and reasonable error handling.
- **Testing** - Meaningful unit and integration tests, independent testing of graph transformations, and sufficient coverage of important edge cases.
- **Scalability and Performance** - Ability to process reasonably large neural network graphs efficiently, avoid unnecessary computation and memory usage, and identify potential performance bottlenecks.
- **Documentation** - Clear installation and usage instructions, explanations of the architecture and key design decisions, and sufficient information to reproduce the provided examples.
- **Problem-Solving and Understanding** - Ability to explain your implementation, justify design decisions, discuss trade-offs, and demonstrate a clear understanding of the submitted code.

> Important: We value correctness, clean architecture, and maintainability over advanced visualization features. A simple, reliable, and extensible solution is preferable to a feature-rich but poorly structured implementation. Optional features are not required for successful completion of the assignment. However, well-designed and properly tested extensions may be considered an advantage. During a follow-up discussion, you may be asked to explain parts of your implementation, discuss alternative approaches, or suggest how your solution could be extended.

7. Use of AI Tools

You are welcome to use AI tools while completing this assignment. However, **you must understand all code you submit**, just as if you had written it yourself. Be prepared to explain your implementation, justify your design decisions, and answer questions about any part of your solution in a follow-up discussion. We value understanding and problem-solving more than the amount of code produced.

8. Deliverables

Please submit a **GitHub repository** containing your implementation and all the materials needed to review and reproduce your results.

The repository should include:

1. **Source code** - A complete implementation of the Python library and command-line tool, including model parsing, the intermediate graph representation, transformation passes, and SVG rendering.
2. **Unit and integration tests** - Tests covering the functionality described in the Testing Requirements section.
3. **Model examples** - Demonstrate your tool using at least three OpenVINO models, including TinyLlama-1.1B-Chat-v1.0.
4. **Generated SVG files** - Provide example visualizations for the selected models, demonstrating the implemented transformation passes. Include before-and-after examples where appropriate.
5. **README.md** - Provide clear documentation covering:
    - **Getting started** - How to install dependencies and run the tool.
    - **Usage examples** - How to use both the CLI and Python API, including how to select and configure transformation passes.
    - **Architecture** - An overview of the main components and key design decisions.
    - **Extensibility** - How to add a new transformation pass or support another model format.
    - **Testing** - How to run the tests.
    - **Performance** - Benchmark results, including graph sizes, execution times, and testing environment.
    - **Known limitations** - Any unsupported cases, performance issues, or possible improvements.

Reproducibility: A reviewer should be able to clone your repository, install the required dependencies, run the tests, and reproduce the provided visualizations by following the README. Large model files do not need to be committed to the repository. Instead, provide instructions or scripts for downloading or exporting them.